alltoall/pairwise

An AgentMPI collective scaling analysis

Abstract

`pairwise` is the default `alltoall` schedule: instead of posting every message at once, rank r performs $p-1$ exchanges, one per round, so that in each round it sends to exactly one peer and receives from exactly one peer. It moves the same $p(p-1)$ messages and the same $p(p-1)n$ tokens as `linear` — 992 messages and 2,976 payload tokens at $p = 32$ — and the sweep confirms both counts at all twenty-one measured sizes, together with the $p-1$ round count at every size including the non-powers of two. What the family view adds is the measured value of the schedule: at $p = 32$ the aggregate time ranks spent blocked in a receive was 30.6 rank-seconds against 250.6 for `linear`, and the spread across ranks was 0.64–1.55 s (a factor of 2.4) against 0.79–22.8 s (a factor of 29). I verified directly from the message log that every one of the 31 rounds at $p = 32$ is a permutation of the ranks, which is the property that produces that balance. One divergence from the documentation is worth recording: the implementation uses the rotation schedule $(r + k) \bmod p$ at every p, including powers of two, and never the XOR schedule its docstring describes, so each round is a single p-cycle rather than a set of mutual pairs, and its freedom from deadlock rests on `sendrecv` posting its receive before its send.

1 What this family is

The `alltoall` collective under the `pairwise` algorithm, measured at 21 process counts from 2 to 32. Every run is agent-free and deterministic, so the measurements are of the protocol itself.

2 What the analysis notices

- [ok] All 21 measured sizes logged exactly the number of messages the closed form predicts.
- [note] Wall times span 0.014–1.135 s with no agent in the loop, so these measure the fabric and the protocol rather than model latency.

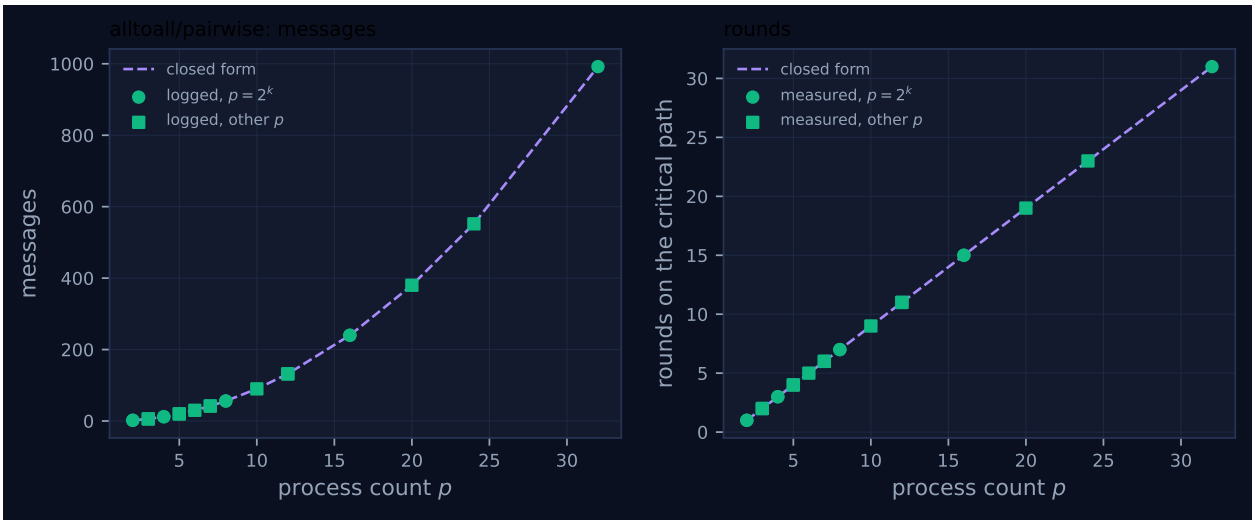

Figure 1: Measured message count and critical-path rounds against process count, with the closed-form prediction. Powers of two are drawn as circles and other sizes as squares, because algorithms take a different code path at sizes that are not powers of two. A red cross marks a size where the collective's own reported count disagrees with the traffic the fabric logged.

3 Every measured size

p	campaign	logged	pred.	self-rep.	rounds	pred.	wall (s)	model	acct.
2	`mb-free`	2	2	2	1	1	0.014	✓	✓
2	`mb-smoke`	2	2	2	1	1	0.014	✓	✓
3	`mb-free`	6	6	6	2	2	0.031	✓	✓
3	`mb-smoke`	6	6	6	2	2	0.019	✓	✓
4	`mb-free`	12	12	12	3	3	0.031	✓	✓
4	`mb-smoke`	12	12	12	3	3	0.046	✓	✓
5	`mb-free`	20	20	20	4	4	0.052	✓	✓
5	`mb-smoke`	20	20	20	4	4	0.052	✓	✓
6	`mb-free`	30	30	30	5	5	0.067	✓	✓
7	`mb-free`	42	42	42	6	6	0.073	✓	✓
7	`mb-smoke`	42	42	42	6	6	0.088	✓	✓
8	`mb-free`	56	56	56	7	7	0.138	✓	✓
8	`mb-smoke`	56	56	56	7	7	0.116	✓	✓
10	`mb-free`	90	90	90	9	9	0.204	✓	✓
12	`mb-free`	132	132	132	11	11	0.205	✓	✓
12	`mb-smoke`	132	132	132	11	11	0.182	✓	✓
16	`mb-free`	240	240	240	15	15	0.274	✓	✓
16	`mb-smoke`	240	240	240	15	15	0.518	✓	✓
20	`mb-free`	380	380	380	19	19	0.596	✓	✓
24	`mb-free`	552	552	552	23	23	0.711	✓	✓
32	`mb-free`	992	992	992	31	31	1.135	✓	✓

4 How the algorithm scales

The cost of `pairwise` is decided by one line: `for k in range(1, p)` with `dst, src = (r + k) % p, (r - k) % p`, and a single `comm.sendrecv` inside. Each rank therefore issues exactly one send and one receive per round and does $p - 1$ rounds, giving $p(p-1)$ messages, $p(p-1)n$ tokens and

$p-1$ rounds on the critical path — which is precisely `FORMULAS["alltoall","pairwise"]`. There is no volume penalty relative to `linear` because nothing is forwarded on anyone else's behalf; the entire difference between the two algorithms is *when* each message is sent.

The two panels of Figure 1 confirm both terms without exception: the message curve is $p(p-1)$ from 2 to 992, and the round curve is the straight line $p-1$ from 1 to 31, with squares and circles alike sitting on it. The round count steps by exactly one at every increment of p and steps nowhere else, because the loop bound is p and contains no logarithm; this is the one `alltoall` algorithm whose round count has no structure to explain. The interesting question is therefore not whether the counts agree — they do, 21 times out of 21 — but what the schedule buys, and for that the closed form is silent and the log is not.

The property the schedule is meant to guarantee is that no rank is oversubscribed in any round. I checked it rather than assuming it: grouping every `msg.send` record by the round index carried in its tag (`_ampi:alltoall:0:1:k`), each round at $p=7$, $p=8$ and $p=32$ contains exactly p edges in which every rank appears once as a source and once as a destination. Round 1 at $p=32$ is the cycle $0\to1$, $1\to2$, $2\to3$, and so on. Each round is a permutation, so the instantaneous fan-in at every rank is one, and the queue of unexpected messages at any destination never exceeds one message per round.

That is visible in the measurements as balance rather than as speed. Aggregate receive-blocking across all ranks grows as $p^{2.85}$ ($R^2=0.993$), reaching 30.6 rank-seconds at $p=32$; the exponent is close to the p^2 of the message count because each individual wait stays short — the longest single receive at $p=32$ was 0.190 s, against 0.865 s under `linear` — and the per-rank totals cluster between 0.64 s and 1.55 s. The viewer screenshot for `mb-free__coll-alltoall-pairwise-16` shows the same thing at $p=16$: every one of the sixteen lanes carries a comparable density of message glyphs spread evenly across the whole 0.3 s span, with no lane empty and no lane privileged, and the collectives panel reports max rounds 15 and 240 messages. Compare the equivalent screenshot for `linear` at $p=32$, where two lanes are blank for most of the run. The wall-clock span is not better — 1.135 s at $p=32$ against 0.959 s for `linear` — because $p-1$ sequential fabric round trips cost more elapsed time than one burst when there is no agent latency to hide. That inversion is a property of the agent-free setting, not of the algorithm.

5 Powers of two and everything else

There is no remainder path, and the reason is worth stating precisely because the docstring implies otherwise. It describes the step-k partner as "`r XOR k` (for powers of two) or `(r+k) mod p` / `(r-k) mod p`". The code implements only the second form, unconditionally. I confirmed this from the log at $p=8$, a power of two: round 1's edges are $0\to1$, $1\to2$, …, $7\to0$, a single 8-cycle. Under an XOR schedule they would have been $0\leftrightarrow1$, $2\leftrightarrow3$, $4\leftrightarrow5$, $6\leftrightarrow7$ — four mutual pairs. So the same code path runs at every p, and the family view shows the expected consequence: the squares at 3, 5, 6, 7, 10, 12, 20 and 24 lie on the closed form as exactly as the circles do, with 6, 20, 30, 42, 90, 132, 380 and 552 messages respectively, and the round count $p-1$ holding at 2, 4, 5, 6, 9, 11, 19 and 23. No red crosses appear, so the self-reported count matched the logged traffic at all twenty-one sizes, and the eight sizes measured twice agree between `mb-free` and `mb-smoke` on every count.

The divergence from the docstring is not a correctness defect but it does move a safety property. A round of mutual XOR pairs is an involution: each exchange involves two ranks and nobody else, so the round completes regardless of whether the underlying primitive sends first or receives

first. A rotation round is a single cycle of length $p/\gcd(p,k)$, and a naive send-then-receive over a cycle is the textbook ring deadlock. This implementation is safe because `Communicator.sendrecv` posts `irecv` *before* calling `send`, which breaks the cycle at every rank simultaneously; the docstring for `sendrecv` says as much, calling itself "the deadlock-free exchange primitive". The practical consequence is that `pairwise`'s freedom from deadlock is a property of the primitive it happens to use rather than of the schedule, and a future refactor that replaced `sendrecv` with an explicit send followed by a receive would deadlock at every $p > 2$ without changing a single count in this table. Either the docstring should be corrected to describe the rotation, or the XOR branch it promises should be implemented.

6 When to choose this algorithm

Within the op family, `pairwise` is the right default and the sweep supports the choice. Against `linear` it is strictly better on every axis that involves a rank waiting: identical message count (992 at $p = 32$), identical payload volume (2,976 tokens), 8.2 times less aggregate blocking, and a bounded fan-in of one message per rank per round instead of $p - 1$ simultaneous unexpected messages. The last of these is the one that decides real deployments, because it is the difference between needing $\Theta(n)$ of receive credit and needing $\Theta(pn)$. The cost is $p - 1$ rounds instead of one, which matters only if the rounds are themselves expensive; in AgentMPI a round is a fabric exchange of an already-computed payload, so the extra rounds cost fabric latency and not agent invocations.

Against `bruck` the trade is genuine and it is rounds against volume. At $p = 32$ `bruck` needs 5 rounds and 160 messages where `pairwise` needs 31 and 992, but it moves 7,680 payload tokens against 2,976 — a factor of 2.6 — because its intermediate messages carry other ranks' data. `cost.predict` makes the comparison explicit: at $p = 32$ with 4,096-token payloads it gives `pairwise` 15.97 s against `bruck`'s 2.58 s on the critical path, and 4.06 M against 10.49 M tokens of volume, so `bruck` is 6.2 times faster and 2.6 times more expensive in tokens. Since the tokens are paid at $15 per million on output, that is $12.19 against $31.46 for one exchange — a real amount of money to spend on latency. `pairwise` is the better choice when volume is what binds, and it has a second advantage the cost model does not price: its messages contain only the sender's own artefact for the named recipient, whereas `bruck`'s intermediate messages carry third parties' artefacts through uninvolved ranks, which is a confidentiality property rather than only a cost.

One thing the cost model gets wrong here is worth flagging, because a harness that asks it will be misled. `cost.best_algorithm("alltoall", p, n, objective="time")` returns `linear` at every (p, n) I evaluated, up to $p = 32$ and $n = 32{,}768$. The reason is in `predict`: the per-message term `message_time` (which is where α lives) is added only when the op is one of `reduce`, `allreduce`, `scan` or `reduce_scatter`, so for `alltoall` the predicted time reduces to rounds $\times$ (fabric $+ n\beta_{\text{in}}$). Under that expression a one-round algorithm always wins, monotonically, no matter how many messages it needs in flight simultaneously. So the model recommends the schedule its own docstring calls "the canonical way to deadlock an AgentMPI program", and it recommends it most strongly exactly where the hazard is worst. The counts in this table are not affected — they come from `FORMULAS`, not from `predict` — but the selection advice built on top of them should not be trusted for `alltoall` until the time expression charges per message.

Zooming out, the choice among the three schedules is a second-order decision inside a first-order problem. Every one of them costs $p(p-1)$ messages or worse, so at $p = 32$ the cheapest full exchange in this repository still charges 160 messages and 992 message-deliveries worth of information flow.

The project's answer is not a better schedule but a sparser graph: in `real-sw-p8-full` the review stage over a degree-2 graph logged 16 messages where a full `alltoall` at $p = 8$ charges 56, and that ratio widens linearly with p. `pairwise` is the algorithm to use when you have decided the full cross-product is genuinely needed; the more valuable decision is usually deciding it is not.

7 Threats to this reading

No agents took part. The executor field is empty, the viewer reports zero agent calls, and the 0.014–1.135 s wall times are fabric writes. The blocking measurements — 30.6 rank-seconds aggregate, 0.190 s worst receive, the $p^{2.85}$ fit — describe SQLite and thread scheduling. They are useful as a *relative* statement against `linear` and `bruck` measured on the same fabric within the same campaign, and they are not evidence about agent latency. In particular the finding that `pairwise` has a longer wall-clock span than `linear` at $p = 32$ (1.135 s against 0.959 s) inverts under any realistic α and should not be quoted as a cost of the algorithm.

The permutation check is structural, not statistical. I verified round-by-round permutation at $p = 7$, $p = 8$ and $p = 32$, not at all thirteen sizes, and the verification recovers the schedule from message tags rather than from the code's intent. A round in which two sends shared a tag would have been counted as one edge; no such collision appeared, but the check is a consistency test rather than a proof.

The deadlock argument is analytic, not measured. Nothing in this sweep exercises the failure I describe, because the runs launched with `eager_limit=10**9` and therefore an 8×10^9-token unexpected budget, under which even `linear` does not stall. The claim that a rotation schedule depends on `sendrecv`'s receive-first ordering follows from reading both implementations; it is a risk to note, not a defect observed.

Two sizes are structurally degenerate. At $p = 2$ the algorithm is one exchange and every schedule coincides, and at $p = 3$ and $p = 4$ the round counts (2 and 3) are too small for the balance argument to distinguish the algorithms. The comparison with `linear` rests mainly on $p \geq 16$, where the sweep has two measurements at $p = 16$ and one each at 20, 24 and 32.